

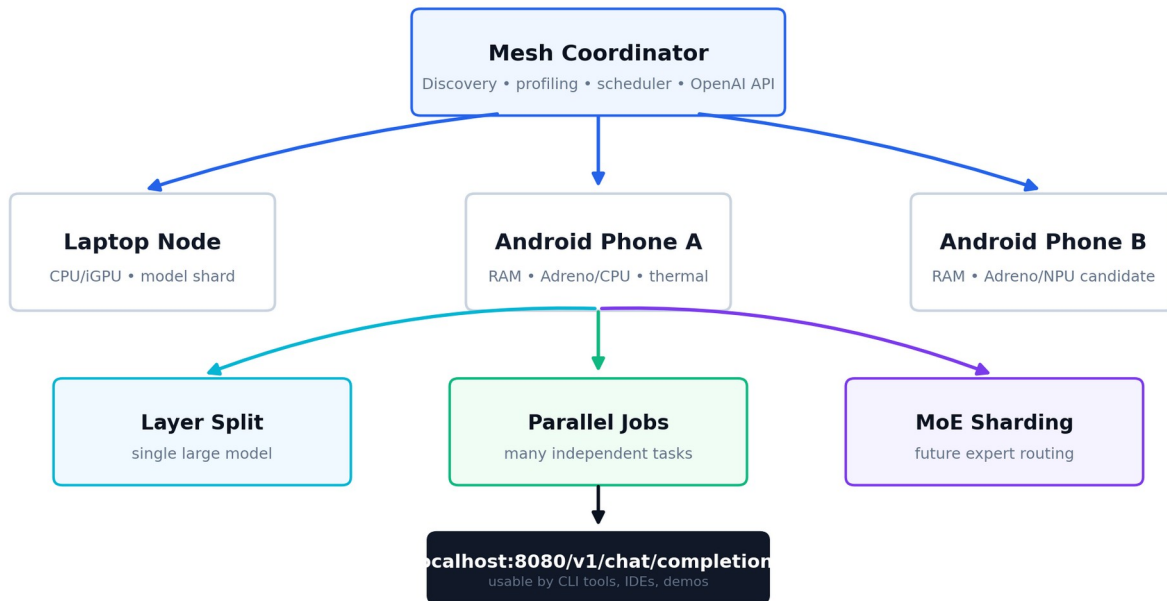
MeshAI

Adaptive Local AI Compute Mesh

Submission Overview + Technical Documentation

MeshAI Architecture: Local AI Compute Mesh

Nearby phones and laptops are profiled, scheduled, and exposed as one local AI endpoint.



One-line thesis

Turn nearby Android phones and laptops into one adaptive local AI machine that can run useful local AI workloads beyond what any single device can handle alone.

Working draft for iQOO Hackathon submission - update URLs, team roster, and benchmark numbers after prototype validation.

1. Executive Summary

MeshAI is a phone-first local AI infrastructure project. It turns nearby Android phones and laptops into a cooperative compute mesh that can run AI workloads locally by automatically discovering devices, profiling hardware, selecting an execution strategy, and exposing the cluster through a normal local AI endpoint.

The project is not positioned as a replacement for ChatGPT or Gemini on raw speed. Cloud AI will usually be faster. MeshAI solves a different problem: users already own multiple capable devices, but their RAM, CPU/GPU/NPU capacity, batteries, and thermal headroom are fragmented. MeshAI makes those devices work together for private, offline, repeated, and cost-sensitive local AI tasks.

For the hackathon MVP, the focus should be a working prototype that proves at least one Android phone and one laptop can participate in the same local AI workload, while the UI shows device discovery, available memory, network quality, temperature, battery state, and the selected execution strategy.

Question	Submission-ready answer
Idea title	MeshAI: Adaptive Local AI Compute Mesh
Core problem	Local AI is limited by the RAM and compute of one device, while phones/laptops nearby sit underused.
Core solution	Automatically pool nearby phones and laptops into a local AI mesh, then select layer splitting, parallel jobs, or future MoE sharding based on workload and device conditions.
Killer demo	Laptop alone runs a smaller/local model. Add phones; MeshAI profiles them and unlocks a larger model, better quantization, larger context, or parallel agent workflow - fully offline.
Novelty focus	Not inventing distributed inference; productizing heterogeneous Android + laptop orchestration with mobile-aware scheduling and a normal local API.

2. Required Submission Fields - Copy-ready Draft

Idea Title

MeshAI: Adaptive Local AI Compute Mesh

Description

MeshAI turns nearby Android phones and laptops into one adaptive local AI compute mesh. Today, a capable local model is usually limited by the RAM, compute and thermal limits of a single device, even though users often have multiple powerful phones and computers sitting idle. MeshAI discovers nearby devices, benchmarks their usable memory, CPU/GPU/NPU capability, network latency, battery level and thermal state, then selects the right execution mode: layer/pipeline splitting for models that do not fit on one device, task parallelism for independent workloads, and future MoE expert sharding for sparse models. The cluster is exposed through a local OpenAI-compatible endpoint so coding tools, chat UIs and document workflows can use it like a normal local model server. The first use case is private/offline coding and document reasoning: a laptop that can only run a smaller model alone can use nearby phones to unlock larger models, higher-quality quantization, larger context, or parallel agents without sending data to the cloud. The phone is not just a screen; it becomes active AI infrastructure.

Video Walkthrough URL

[TBD after prototype] Recommended: 2-3 minute unlisted YouTube/Drive video showing laptop-only baseline, phone joining, live profiling, selected strategy, offline inference, and final output.

Prototype URL

[TBD after prototype] Recommended: GitHub repository + release APK/desktop coordinator + README + benchmark table. If hosted, use a landing URL that links to the repo, APK, and demo video.

Deck/Document

Use this document as the supporting submission document. After prototype validation, create a shorter 8-10 slide deck using the same diagrams and measured results.

Android Proficiency

Our team is comfortable building Android-first products across UI, device APIs, local storage, networking and native integration. We have experience with Flutter/Android apps, local-first SQLite/JSON storage, Firestore sync, notifications, health/sensor integrations, maps/payments, and device-centric AI experiences. For MeshAI, the Android work requires a foreground worker service, native llama.cpp/NDK integration, device profiling, thermal and battery monitoring, Wi-Fi/LAN or Wi-Fi Direct discovery, APK packaging and on-device testing. This maps directly to our prior mobile work and to the project's phone-first requirement.

LLM Proficiency

Our team has worked with AI pipelines involving local and cloud LLMs, STT/LLM/TTS orchestration, RAG-style knowledge handling, multi-provider model routing, tool/API workflows and local inference research. For this project, we have specifically studied llama.cpp/GGUF, RPC-based distributed inference, Android builds, OpenAI-compatible local endpoints, coding-agent integrations and device-aware scheduling. We understand that the value is not simply "using an LLM API"; the technical challenge is model placement, KV cache management, network boundaries, quantization, context size, accelerator selection and mobile thermal constraints.

Prior Builds and Hackathons

Selected prior builds include AI voice/calling systems, Flutter/Android health apps with local-first data and AI chat, pose/fitness tracking systems, finance/AI application concepts, and full-stack mobile/backend workflows involving auth, payments, maps, real-time sync and admin dashboards. These builds show that the team can connect mobile UI, backend systems, AI models and hardware-aware constraints. Verified hackathon participation/awards should be inserted here before final submission if applicable; do not overclaim unverified placements.

What Makes You and Your Team Stand Out?

We are not building another chatbot wrapper. We are attacking the systems layer underneath local AI: how phones, laptops and desktops can cooperate as one adaptive AI computer. The team's strength is full-stack execution across mobile apps, native/device integration, AI model workflows and backend systems. This lets us build the Android worker, desktop coordinator, scheduler, local API and demo workflow ourselves instead of relying only on external APIs. MeshAI also fits the hackathon unusually well because the phone is not secondary - it becomes an active compute node, measured and used through RAM, battery, thermals, network and accelerator capability.

3. Problem and Product Positioning

Problem	Why it matters	MeshAI answer
Single-device ceiling	An ordinary laptop or phone often cannot fit a larger local model or a useful context window.	Pool nearby devices and select the best model placement or task-parallel strategy.
Cloud dependency	Cloud AI is fast, but private repos/docs, offline environments and repeated inference can be difficult or costly.	Local mesh inference keeps data inside user-controlled devices.
Unused hardware	Phones and laptops are usually idle while a single device struggles with AI workloads.	Use phones as active compute nodes, not just screens.
Mobile constraints	Phones throttle, lose battery, sleep, or join/leave the network.	Scheduler monitors thermal, battery, link quality and rejects devices that hurt performance.

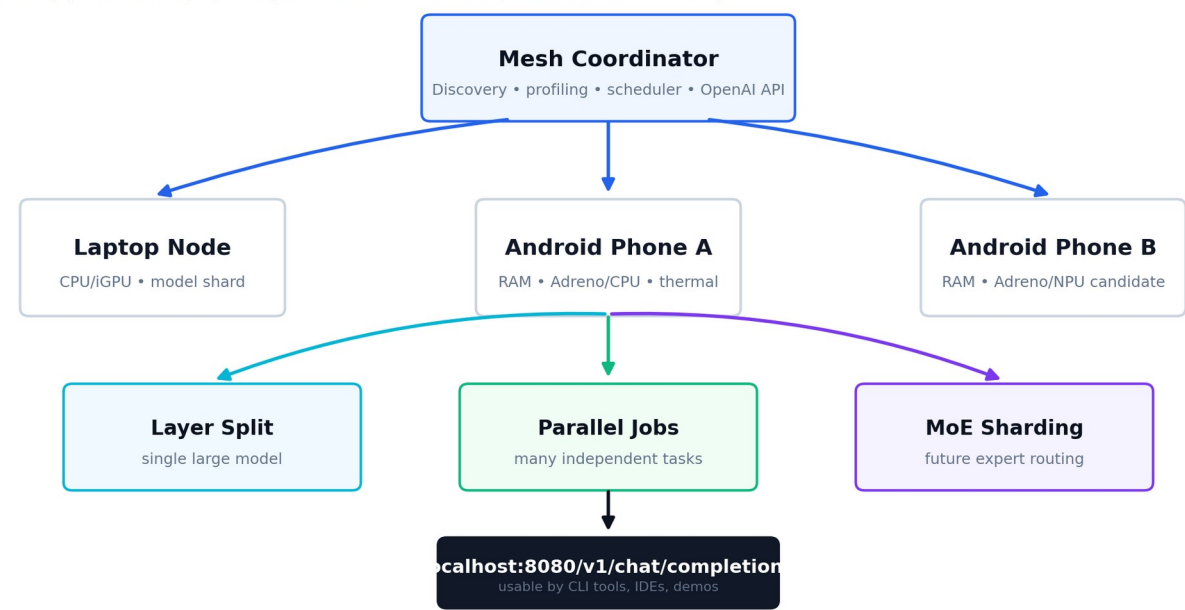
Positioning rule
Do not claim MeshAI beats cloud frontier models on speed. The stronger claim is: MeshAI unlocks private, offline, adaptive local AI from hardware users already own.

4. System Architecture

MeshAI has three roles: a coordinator, worker nodes, and a scheduler. The coordinator may run on a laptop or capable phone. Worker nodes run on Android phones and other computers. The scheduler decides whether a device should participate and how.

MeshAI Architecture: Local AI Compute Mesh

Nearby phones and laptops are profiled, scheduled, and exposed as one local AI endpoint.



Component	Responsibility	MVP implementation
Coordinator	Owns model session, device registry, benchmark results, model plan and local API.	Desktop Python/Go/Node app or local server wrapping llama.cpp.

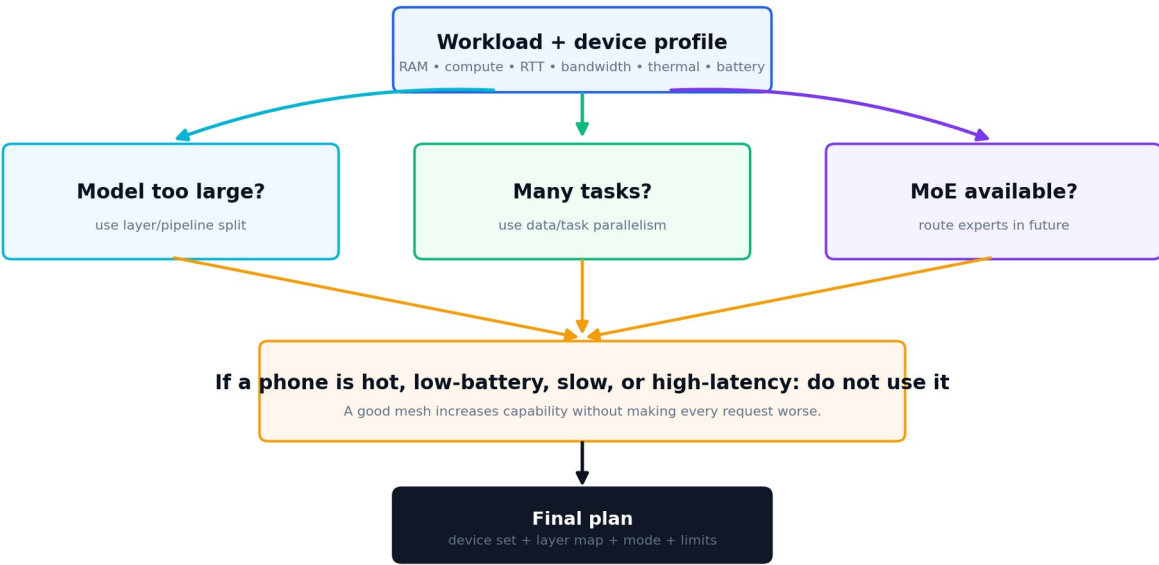
Component	Responsibility	MVP implementation
Android worker	Reports memory/thermal/battery/network status; exposes local inference backend.	Kotlin/Compose shell + native llama.cpp/ggml-rpc-server via NDK or Termux-first prototype.
Scheduler	Chooses layer split, parallel jobs, MoE route or reject-device decision.	Start with memory + benchmark + RTT heuristic; improve after measurements.
Dashboard	Shows topology, layer map, mode, tokens/sec, TTFT, RAM, battery and thermals.	Local web UI or Android + desktop UI.
API endpoint	Allows existing tools to use the mesh.	OpenAI-compatible local endpoint: /v1/chat/completions.

5. Inference Strategies

MeshAI should support multiple strategies because one strategy will not be optimal for every workload or network. The product value is the AUTO scheduler: it chooses the best strategy from measured device conditions.

AUTO Scheduler: Split, Parallelize, or Reject

The system should not blindly use every device. It chooses a strategy from workload + device + network signals.



Strategy	How it works	When to use	Risk
Layer / pipeline split	Contiguous transformer layers are placed on different devices. Activations move at device boundaries; weights and KV cache stay local to each assigned node.	One model is too large for one device, and network boundary cost is acceptable.	Can be slower than single-device inference if network latency dominates.
Data / task parallelism	Each device runs an independent full task or agent: reviewer, searcher, test analyzer, document chunker, image variant	Many independent jobs or coding-agent subtasks.	Does not increase max single-model size.

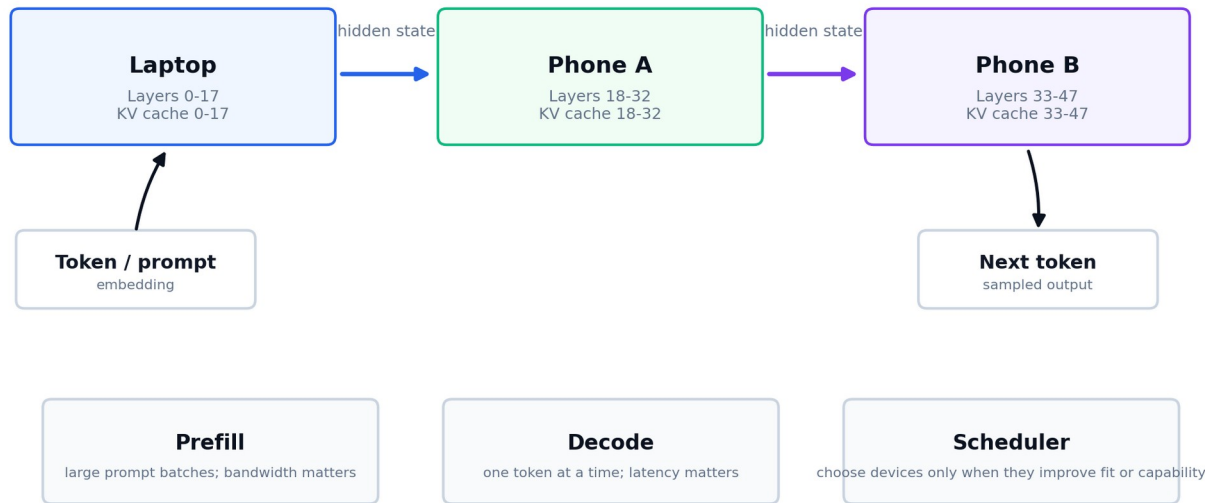
Strategy	How it works	When to use	Risk
MoE expert sharding	Future sparse models store different experts on different nodes; router calls only required experts.	MoE models where expert routing reduces communication.	More complex runtime; prototype after layer/task modes.
Tensor parallelism	Devices split the math inside the same layer.	Fast wired interconnects such as PCIe/NVLink/RDMA-like links.	Poor first choice over normal Wi-Fi due to frequent synchronization.

6. What Happens Between Layers

A transformer model is a sequence of blocks. For a model with 48 layers, the output of layer 17 can become the input to layer 18 on another device. The entire weight tensor does not move every token; the next device receives an activation/hidden-state tensor. Each device also keeps the KV cache for the layers it owns.

Layer Split: What Moves Between Devices

Weights stay resident on each node; boundary activations move forward, while each node keeps KV cache for its own layers.



Term	Meaning in MeshAI
Weights	Model parameters. They should be loaded or cached on the device assigned to those layers. They should not be retransferred every token.
Activation / hidden state	The intermediate tensor passed from one layer block to the next. This is what moves across device boundaries during pipeline splitting.
KV cache	Per-layer conversation memory used during attention. In layer split mode, KV cache should stay with the same device that owns those layers.
Prefill	Initial prompt processing. Large prompts create bigger boundary tensors, so bandwidth matters more.
Decode	One-token-at-a-time generation. Boundary payloads are smaller, but latency/synchronization matters more.

Concrete model reference

Qwen3-Coder-30B-A3B has 48 hidden layers, hidden size 2048, 32 attention heads, 4 KV heads, 128 experts and 8 experts per token. This makes it a useful technical reference for discussing layer count, KV cache and MoE routing.

7. Technical Foundation and Existing Work

The project should be honest about what exists. We are not inventing distributed inference; we are building a polished heterogeneous Android + laptop orchestration layer on top of existing primitives.

Existing system	What it proves	Gap MeshAI targets
llama.cpp RPC	Remote ggml devices can participate in distributed inference; model weights and KV cache can be distributed across local and remote devices.	RPC is proof-of-concept, fragile/insecure for open networks and lacks consumer discovery, scheduler and phone UX.
llama.cpp Android	Android arm64 builds and app bindings exist; models can run locally on Android.	Need a clean worker app plus device profiling and orchestration.
AI Doomsday Toolbox	Android phone clusters and RAM-aware layer splitting have been demonstrated in an Android app.	Manual/experimental; not positioned as heterogeneous laptop+Android automatic scheduler.
Exo	Excellent automatic discovery, topology-aware parallelism and polished cluster concept.	Current public focus is Apple/Mac and desktop-class clusters, not Android phone + laptop as primary target.
Synapse	Cross-platform browser/WebGPU layer splitting across phones works; published phone demos show the concept is real.	Browser prototype performance is low; native Android/laptop orchestration can target practical workflows.
Meshlit	Android federated inference, data-parallel jobs and MoE sharding are active directions.	It explicitly avoids dense layer splitting across phones; MeshAI can test when layer split is still worthwhile and choose automatically.

8. iQOO Hackathon Fit

The iQOO hackathon scoring is unusually aligned with this project. The official guide awards 30% for end-product quality, 20% for novelty/impact, 15% for creative phone use, 15% for technical depth, 10% for Office Kit usage and 10% for demo/presentation. It also requires phone-first execution and states that local/open-source models with the phone in the loop are rewarded.

Scoring dimension	How MeshAI addresses it
End product quality - 30%	A clear dashboard, reliable worker app, benchmark panel and local API endpoint create a usable product rather than a research script.
Novelty/impact - 20%	The novelty is consumer-grade heterogeneous Android + laptop orchestration, not another AI app wrapper.
Creative phone use - 15%	The phone is an active AI compute node; its RAM, battery, thermals and network state affect the model plan.
Technical depth - 15%	Layer placement, KV cache, activation transfer, runtime backends, thermal scheduling and strategy selection are real systems problems.
Office Kit usage - 10%	Use the iQOO + laptop bridge to demonstrate laptop coordinator + phone worker and show HackTracker-visible usage.
Demo/presentation - 10%	The demo is physical: connect phones, capability unlocks,

Scoring dimension	How MeshAI addresses it
	internet off, task runs locally.

9. Prototype Plan

The prototype should be built from smallest proof to polished demo. Do not start with a 30B model. First prove laptop-to-Android split on a tiny model, then scale.

Stage	Goal	Success condition
0. Research lock	Confirm exact hardware, model files and runtime path.	Device list, RAM, OS, network and target models documented.
1. Bare split proof	Laptop + one Android phone run different parts of a small GGUF model.	Logs prove phone executes assigned layers; output is valid.
2. Backend benchmark	Compare CPU, Adreno/OpenCL/Vulkan, and possible Hexagon path.	Choose default Android backend from measured tok/s, TTFT, thermals and reliability.
3. Two-phone mesh	Add second Android phone and run larger/parallel workload.	Dashboard shows both phones and scheduler plan.
4. Product demo	Create UI + OpenAI-compatible endpoint + offline task demo.	Video shows laptop baseline, phones joining, strategy selection and local output.

Metric to capture	Why it matters
TTFT	Shows startup and prompt-processing responsiveness.
Decode tokens/sec	Shows live generation speed.
Prompt tokens/sec	Shows prefill/bandwidth impact.
RAM per node	Shows actual pooled capacity and whether larger model/context is realistic.
Network throughput + RTT	Determines whether layer boundaries are viable.
Temperature + thermal headroom	Determines whether phone workload is sustainable.
Battery drain / charging state	Protects user devices and guides scheduler policy.

10. Recommended Demo Flow

- Open MeshAI dashboard on laptop. Show no internet dependency.
- Run baseline: laptop-only model or agent; show current model limit and metrics.
- Open Android worker app on iQOO/phone. Tap Join Mesh. Repeat with second phone if available.
- Dashboard benchmarks devices: usable RAM, compute score, RTT, bandwidth, battery, temperature.
- Scheduler selects a plan: layer split for a larger model, or parallel agents for a coding/document task.
- Run task: private repo analysis, document Q&A, or coding bug-fix prompt through local endpoint.
- Show physical proof: phone metrics change, assigned layers/jobs visible, tokens stream locally.
- Switch off internet or show airplane-mode/local network setup; inference continues.

Demo line

Your devices are individually too weak to run this AI workload comfortably. MeshAI turns them into one adaptive local AI machine.

11. Risks, Mitigations and Claims Discipline

Risk	Mitigation
Layer split is slower than laptop-only	AUTO scheduler should reject slow devices or use parallel-agent mode instead.
Android accelerator path is unstable	Start with CPU worker; add Adreno as target; treat Hexagon NPU as advanced bonus.
RPC is insecure/fragile	Use trusted private LAN for hackathon; later add QR pairing, authentication and encrypted transport.
Hot-plug migration is hard	For MVP, rebalance between requests or pause/restart generation with context restored.
30B/70B claims overpromised	Use measured benchmarks only. Phrase as “larger/better local workloads can be unlocked” until numbers are proven.
Cloud is faster	Position around privacy, offline, cost, repeated local inference and hardware reuse - not cloud speed replacement.

12. What Still Must Be Researched or Tested Before Final Submission

- Run the laptop + Android RPC kill-test using a tiny GGUF model.
- Measure one-device vs mesh performance using the same prompt and model.
- Confirm whether Adreno/OpenCL works cleanly inside the Android worker in our deployment path.
- Try Hexagon only after CPU/Adreno is stable; do not make the NPU the MVP dependency.
- Decide final demo workload: coding CLI, repo analysis, document Q&A, or parallel-agent workflow.
- Finalize team facts: names, roles, exact prior builds, verified hackathon history and final URLs.

Decision gate

If layer splitting is too slow on real hardware, keep the same product but emphasize parallel agents and MoE-shard-ready scheduling. The UI, discovery, profiling and orchestration remain valuable.

13. Sources and Technical References

- **iQOO Hackathon 2026 guide and scoring:** <https://iqoo.reskilll.com/guide>
- **iQOO 15 specifications:** <https://www.iqoo.com/in/products/param/iqoo15>
- **llama.cpp RPC backend README:** <https://github.com/ggml-org/llama.cpp/blob/master/tools/rpc/README.md>
- **llama.cpp Android documentation:** <https://github.com/ggml-org/llama.cpp/blob/master/docs/android.md>
- **llama.cpp server options / RPC and KV cache options:** <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- **llama.cpp Snapdragon Hexagon backend developer details:** <https://github.com/ggml-org/llama.cpp/blob/master/docs/backend/snapdragon/developer.md>
- **llama.cpp OpenCL backend / Android notes:** <https://github.com/ggml-org/llama.cpp/blob/master/docs/backend/OPENCL.md>
- **Android Wi-Fi Direct service discovery:** <https://developer.android.com/develop/connectivity/wifi/nsd-wifi-direct>
- **Qwen3-Coder-30B-A3B config:** <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct/blob/main/config.json>
- **Exo distributed AI cluster:** <https://github.com/exo-explore/exo>
- **AI Doomsday Toolbox:** <https://github.com/ManuXD32/AI-Doomsday-Toolbox>
- **Synapse distributed browser LLM:** <https://github.com/tejasphatak/Synapse>
- **Meshlit Android federated inference:** <https://github.com/sabbirimon/meshlit>